

Provider–Observer–Operator: Interface-Led Architecture における役割ベース抽象化規律

Takuya Sogawa

目次

Provider–Observer–Operator	1
Interface-Led Architecture における役割ベース抽象化規律	1
Abstract	1
Keywords	2
1. Introduction	2
2. Interface-Led Architecture との関係	2
3. Problem Statement	3
4. Primitive Role Model	3
5. Unit as Governed Composition	5
6. Pipeline and Core Contract	5
7. Exception as State, Not Role	6
8. Optional Metadata Rule	6
9. Relationship to Reactive Architecture	7
10. General ILA Core and AIKernel Specialization	7
11. Axioms of ILA Abstraction	8
12. Practical Refactoring Guidelines	8
13. Limitations and Non-Claims	8
14. Conclusion	9
References	9

Provider–Observer–Operator

Interface-Led Architecture における役割ベース抽象化規律

Version: v1.0.0

Type: Technical Note / ILA Supplement

Author: Takuya Sogawa

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: 10.5281/zenodo.20322690

License: CC BY 4.0

Abstract

本稿は、Interface-Led Architecture (ILA) を実装設計へ適用するための補助理論として、Provider・Observer・Operator に基づく役割ベース抽象化規律を定義する。

ILA は、ソフトウェアを実装クラス、フレームワーク層、継承階層、または具体的な技術スタックによって抽象化しない。代わりに、構成要素を Provider、Observer、Operator という三つの基本役割へ分類し、

それらの合成体を Unit として扱う。Unit は Core Contract の上で Pipeline によって連結され、ソフトウェア構造は役割と契約の合成として再構成される。

本稿の中心命題は、インタフェース設計において抽象化すべき対象は「再利用される実装」ではなく、「供給される能力」「観測される状態」「実行される操作」、およびそれらを接続する契約境界である、という点にある。Provider は能力を供給する。Observer は証拠を観測する。Operator は状態を変換または実行する。Unit は役割を合成する。Pipeline は Unit を順序づける。Core Contract はそれらの相互作用を制約する。

また、本稿は例外を役割として扱わない。例外は Provider、Observer、Operator のいずれにも属さず、Pipeline が Core Contract の下で継続不能になったときに発生する失敗状態または遷移結果として位置づけられる。契約上の正しさに不要な追加情報はすべて optional metadata として扱われる。

Semantic IR、Composite Distance、Governed Circuits、ReplayLog は ILA の一般理論ではない。それらは、ILA を AIKernel に適用した場合に現れる特殊化である。この分離により、ILA の一般性を保ちながら、AIKernel の semantic runtime governance への接続を維持する。

Keywords

Interface-Led Architecture; ILA; Provider; Observer; Operator; Unit; Pipeline; Core Contract; Role-Based Abstraction; Software Architecture; AIKernel; Contract-First Design; Fail-Closed; Capability-Based Design

1. Introduction

Interface-Led Architecture (ILA) は、実装よりも先にインタフェースを定義し、ソフトウェア構造を契約境界から構成するための設計方法論である。その目的は、単に再利用性を高めることではなく、アーキテクチャ上の境界を明示し、検査可能で、強制可能な形へ変換することにある。

しかし、ILA を実際のコードベースへ適用する際には、実践上の問いが発生する。すなわち、**インタフェースはどのように発見されるべきか**、という問いである。

この問いに対する答えが、実装の共通性、フレームワーク層、または将来の再利用可能性だけに基づく場合、抽象化は不安定になりやすい。巨大なインタフェースが生まれ、未対応メソッドが通常設計に入り込み、Provider にポリシー判断が混入し、Observer が状態を書き換え、例外が通常の制御フローになり、optional metadata が暗黙の必須情報になってしまう。

本稿は、この問題を避けるための小さな抽象化規律を定義する。すなわち、インタフェース抽出は、まず構成要素を次の三つの役割へ分類することから始まる。

- **Provider** —何かを供給する
- **Observer** —何かを観測する
- **Operator** —何かに作用する

上位構造は、新しい基本役割ではない。それらは三役の統治された合成である。本稿ではこの合成体を **Unit** と呼ぶ。Unit は **Pipeline** によって連結され、Pipeline は **Core Contract** によって定義された遷移の上でのみ正当化される。

2. Interface-Led Architecture との関係

本稿は、既存の Interface-Led Architecture 方法論を補足するものである。ILA 論文は、ソフトウェアを実装優先ではなく、インタフェース契約、責務境界、決定論的実行文脈から構成するという大きな設計姿勢を定義した。

本稿が扱う問いは、より狭い。すなわち、**そのインタフェースを抽出するための抽象化規律は何か**、である。

その関係は次のように整理できる。

Interface-Led Architecture:

インタフェース、契約、責務境界からソフトウェアを構成する。

Provider-Observer-Operator Discipline:

構成要素を基本役割へ分類し、Unit として合成し、
Pipeline によって接続することでインタフェースを抽出する。

したがって、本稿は ILA を置き換えるものではない。ILA をリファクタリングと実装設計へ適用可能にするための補助理論である。

3. Problem Statement

一般的な抽象化は、しばしば誤った類似性から始まる。二つのクラスが実装を共有しているか、同じフレームワーク層に属しているか、将来再利用されそうか、といった観点である。これらは有用ではあるが、インタフェース設計の基準としては不十分である。

実装上似ている二つの構成要素が、契約上同じ責務を持つとは限らない。逆に、実装が異なっても、契約上は同じ役割を持つ場合がある。

たとえば、あるファイルコンポーネントが、読み取り、書き込み、削除、列挙、観測、スナップショット、監査を行う場合、それらを単一の巨大な `IFileSystem` としてまとめると、読み取り専用 **Provider** にも不要な操作が契約上露出してしまう。その結果、システムは契約による分離ではなく、実行時拒否に依存する。

典型的な症状は次の通りである。

- **Interface** が過剰に大きい
- `NotSupportedException` が通常設計に入り込む
- **Provider** がポリシー判断を含む
- **Observer** が状態を変更する
- **Operator** が optional metadata に暗黙依存する
- 例外処理が通常制御フローとして使われる
- **Pipeline** が Core Contract ではなく実装知識によって結合される

ILA には、実装詳細より先に、役割、契約、合成を分離する規律が必要である。

4. Primitive Role Model

ILA における最小の抽象化単位は、クラス、パッケージ、モジュール、フレームワーク層ではない。**Role** である。

Role は、ある構成要素がソフトウェアシステム内で果たす契約上の責務を表す。インタフェース抽出の観点から、ILA は **Role** を次の三つの基本役割へ分類する。

Role ::= Provider | Observer | Operator

これは、世界のすべての存在が存在論的にこの三種類へ還元される、という主張ではない。主張はより限定的である。すなわち、インタフェース抽出と責務分離の観点では、この三役が十分な基本単位である、ということである。

4.1 Provider

Provider は、データ、状態、能力、資源アクセス、サービス、または実行対象を契約に従って供給する役割である。

Provider は「何を提供できるか」を宣言する。上位のアクションが安全か、認可されているか、目標に整合しているか、意味的に許容されるか、といった判断は行わない。これらは **Policy**、**Gate**、**Operator**、または上位 **Unit** の責務である。

例:

MemoryProvider
ModelProvider
StateProvider
VfsProvider
EmbeddingProvider
ClockProvider
ReplayLogProvider

Provider の基本規則は次の通りである。

Provider は能力を供給する。
Provider は統治判断をしない。

4.2 Observer

Observer は、状態、変化、メトリクス、リスク、逸脱、イベント、ログ、または証拠を観測する役割である。

Observer は証拠を生成する。観測対象のシステム状態を直接変更してはならない。Observer は、Policy Decision Point、Operator、ReplayLog、診断、監査システムへ情報を提供できるが、観測と変更は分離されるべきである。

例:

StateObserver
RuntimeObserver
TrajectoryObserver
RiskObserver
DriftObserver
ReplayObserver
MetricObserver

Observer の基本規則は次の通りである。

Observer は証拠を観測する。
Observer は状態を変更しない。

4.3 Operator

Operator は、入力、状態、資源、または意味構造に対して作用する役割である。読み取り、書き込み、実行、推論、変換、検証、コンパイル、合成、射影、ルーティング、判定はすべて Operator に属する。

ILA において、「使う」「実行する」「変換する」「判定する」はすべて操作である。Operator は状態遷移や副作用を引き起こす可能性があるため、Provider や Observer より強い契約境界を必要とする。

例:

ReadOperator
WriteOperator
InferenceOperator
TransformOperator
ValidationOperator
SynthesisOperator
AdmissibilityGate
PolicyDecisionOperator

Operator の基本規則は次の通りである。

Operator は状態を変換する。
Operator は統治されなければならない。

5. Unit as Governed Composition

上位構造は第四の基本役割ではない。Runtime、Layer、Pipeline、Module、Subsystem、Serviceなどは、多くの場合 Provider、Observer、Operator の合成体として理解できる。

本稿では、この統治された合成体を **Unit** と呼ぶ。

Unit = Provider / Observer / Operator の統治された合成体

Unit は Pipeline に参加できる最小の意味的まとまりである。

5.1 例

MemoryUnit

```
├ Provider: MemoryProvider
└ Operator: Read / Write / Snapshot
```

RuntimeUnit

```
├ Provider: RuntimeStateProvider
├ Observer: RuntimeObserver
└ Operator: ExecutionOperator
```

PipelineUnit

```
├ Observer: PipelineStateObserver
└ Operator: PipelineStepOperator
```

ModelUnit

```
├ Provider: ModelProvider
├ Observer: TokenUsageObserver / LatencyObserver
└ Operator: InferenceOperator
```

AIKernel における VFS Unit は次のように表現できる。

VFS Unit

```
├ Provider: IVfsProvider
├ Observer: IVfsStateObserver / IVfsAuditObserver
└ Operator: IReadableVfsSession / IWritableVfsSession
```

Trajectory Governance Unit は次のように表現できる。

Trajectory Governance Unit

```
├ Provider: CandidateProvider / ContextProvider
├ Observer: TrajectoryObserver / RiskObserver
└ Operator: TrajectoryGovernor / PolicyDecisionOperator
```

5.2 Unit の性質

Unit は以下の性質を持つ。

- 基本役割の合成によって定義される
- 実装よりも Interface / Contract を優先する
- Core Contract の上で動作する
- 他の Unit と Pipeline によって接続される
- optional metadata を公開してもよい
- optional metadata は契約上の正しさに必須であってはならない

6. Pipeline and Core Contract

Pipeline は Unit の順序付き合成である。

Pipeline = Unit_1 -> Unit_2 -> ... -> Unit_n

ただし、Pipeline は単なるメソッド呼び出し列ではない。ILA において Pipeline が正当であるためには、各遷移が **Core Contract** によって定義されていなければならない。

Core Contract は、Unit 間の遷移に必要な最小の契約境界である。少なくとも以下を含む。

- Input contract
- Output contract
- Capability declaration
- Failure mode
- Determinism requirement
- Security boundary
- Replay requirement

AIKernel に適用すると、次のような構造になる。

```
ROM Unit
-> VFS Unit
-> Pre-Inference Governance Unit
-> Trajectory Governance Unit
-> Execution Unit
-> Replay Unit
```

各 Unit は、次の Unit の実装を知る必要はない。Pipeline が要求する Core Contract を満たせばよい。

7. Exception as State, Not Role

例外は役割ではない。

例外は Provider、Observer、Operator のいずれにも属さない。例外は、Pipeline が Core Contract の下で継続不能になったときに発生する失敗状態または遷移結果である。

Exception is not a role.

Exception is a failure state of a Pipeline or Contract transition.

この区別は重要である。例外を通常制御フローとして扱うと、アーキテクチャは契約上の妥当性ではなく、実行時の逃げ道に依存し始める。ILA において、失敗は可能な限り契約上の状態として表現されるべきである。

AIKernel において、この考え方は Fail-Closed 設計と整合する。Pipeline が安全に継続できない場合、システムは失敗状態を記録し、拒否、明確化要求、中止、または安全なフォールバックへ遷移する。

8. Optional Metadata Rule

契約に含まれない追加情報はすべて任意である。

Core Contract は、正しさに必要な情報だけを含むべきである。診断、文書化、ルーティングヒント、UI、観測性、Provider 固有の最適化などに役立つ情報は存在してよいが、契約上の妥当性に必須であってはならない。ただし、その情報が正しさに必要である場合、それは metadata ではなく Contract に昇格されるべきである。

必須情報の例:

```
Role
Input
Output
Capability
Failure mode
Contract boundary
```

任意情報の例:

description
tags
display name
diagnostics
statistics
provider-specific hints
UI hints
documentation metadata

metadata が Pipeline の正しさに必要であるなら、それはもはや metadata ではない。Core Contract の一部である。

9. Relationship to Reactive Architecture

Reactive アーキテクチャは、観測、伝播、非同期イベント処理、変化への応答を重視する。この視点は、観測性と動的伝播の重要性を明確にした点で価値がある。

ILA は Reactive アーキテクチャを否定しない。むしろ、Reactive が前景化した Observer 中心の視点を、Provider、Observer、Operator を分離した役割完全な抽象化モデルへ一般化する。

Reactive architecture:
観測と伝播が強調される。

ILA role model:
Provider、Observer、Operator が基本役割として分離される。

この意味で、ILA は Reactive の強みを吸収しつつ、アーキテクチャ全体を観測だけに還元しない。

10. General ILA Core and AIKernel Specialization

Provider-Observer-Operator モデルは ILA の一般理論に属する。

ILA General Core
├ Provider
├ Observer
├ Operator
├ Unit
├ Pipeline
└ Core Contract

Semantic IR、Composite Distance、Governed Circuits、ReplayLog、Semantic State、Runtime Policy は ILA の一般理論ではない。これらは、ILA を AIKernel と semantic runtime governance へ適用したときに現れる特殊化である。

AIKernel Specialization
├ Semantic IR
├ Composite Distance
├ Governed Circuit
├ ReplayLog
├ Semantic State
└ Runtime Policy

この分離は、ILA の一般性を守る。Semantic IR や Composite Distance を ILA そのものに含めてしまうと、ILA は AIKernel 専用の理論になってしまう。ILA は一般的なインタフェース設計方法論であり、AIKernel はその semantic runtime governance への応用例である。

11. Axioms of ILA Abstraction

本稿の抽象化規律は、次の公理として要約できる。

Axiom 1: Primitive Role Classification

インタフェースを持つ構成要素は、Provider、Observer、Operator、またはそれらの合成体である Unit として分類される。

Axiom 2: Unit Is Not a Fourth Primitive Role

Unit は第四の基本役割ではない。Unit は基本役割の統治された合成体である。

Axiom 3: Pipeline Is Ordered Unit Composition

Pipeline は、Core Contract の下で動作する Unit の順序付き合成である。

Axiom 4: Exception Is Not a Role

例外は役割ではない。例外は Pipeline または Contract 遷移における失敗状態である。

Axiom 5: Optional Metadata Must Remain Optional

追加情報は、契約上の正しさに必要でない限り任意である。正しさに必要である場合、それは Core Contract へ昇格されなければならない。

12. Practical Refactoring Guidelines

既存実装を ILA に基づいて整理する場合、次の問いを順に適用する。

1. この構成要素は何を供給しているか
2. この構成要素は何を観測しているか
3. この構成要素は何を操作、変換、実行、または判定しているか
4. これは基本役割か、それとも複数役割から成る Unit か
5. Unit 間の遷移は Core Contract によって表現されているか
6. Provider にポリシー判断が混入していないか
7. Observer が状態を変更していないか
8. Operator は明示的な契約によって統治されているか
9. 例外を通常制御フローとして使っていないか
10. optional metadata が暗黙の必須情報になっていないか
11. NotSupportedException を通常パスとして要求していないか
12. Pipeline は Fail-Closed 可能か

NotSupportedException が Interface 実装に通常パスとして現れる場合、それは契約が大きすぎるか、役割分離が不十分である可能性が高い。

13. Limitations and Non-Claims

本稿は、ソフトウェア、計算、または世界のすべての存在が、存在論的に Provider、Observer、Operator へ還元されると主張するものではない。

本稿の主張は、アーキテクチャ上・方法論上のものである。すなわち、インタフェース抽出と責務分離の観点において、この三役は契約設計のための簡潔かつ十分な基盤を提供する。

また、本稿は Semantic IR、Composite Distance、Semantic Ellipsoid、Governed Circuits、ReplayLog を ILA の一般概念として定義しない。これらは AIKernel への応用に属する。

さらに、本稿は Reactive architecture を否定しない。Reactive systems が強調した Observer 中心の洞察を、供給と操作を含むより広い役割モデルの中へ再配置する。

14. Conclusion

本稿は、Interface-Led Architecture のための役割ベース抽象化規律として、Provider–Observer–Operator モデルを定義した。

ILA のインタフェース設計は、Provider、Observer、Operator という三つの基本役割から始まる。それらの統治された合成体が Unit である。Unit は Pipeline によって連結される。Pipeline は Core Contract の上で動作する。追加情報は、契約上の正しさに必要でない限り任意である。

例外は役割ではない。例外は Pipeline が Core Contract の下で継続不能になったときの失敗状態である。

Semantic IR や Composite Distance は ILA の一般理論ではなく、AIKernel への応用時に現れる特殊化である。

この規律により、ILA は単なる interface-first design の思想ではなく、ソフトウェアシステムを Role、Unit、Pipeline、Core Contract へ再構成するための実践的なリファクタリング方法論となる。

References

1. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.”Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
2. Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
3. Bonér, Jonas, Dave Farley, Roland Kuhn, and Martin Thompson. “The Reactive Manifesto.”2014. Available at: <https://www.reactivemanifesto.org/>.